NIM sizing: from request to recommendation

Implemented API + CLI flow | Current PR behavior | No deployment or reservation

1 Request, authorization and cluster inventory

1 API or CLI request

Model image + exact tag + clusterId
Optional gpuType / --gpu-type
CLI uses the selected cluster context

2 Authorize + validate

Require inference-create permission
Validate cluster, image, tag and type
Rejected requests stop here

3 Read + select inventory

Read every authorized node-pool page
Exact GPU type, or all type/capacity pairs
Empty inventory + all types: 200, results []

Selection nonempty

Continue from right to left

2 Fetch profiles, select a published layout, size the allocation

4 Read cached / NGC data

Exact image/tag cache; expiry: 15 min
Miss / cache failure: query NGC
Cache profiles only; skip empty / errors

5 Select a fitting published profile

Per-device memory + one-node count fit
TP1 first; direct > tested > GPU-agnostic
Then fewest GPUs, minimum, profile ID

6 Allocate selected layout

TP1 below 90%: published memory
TP1 at 90–100%: one whole GPU
Parallel profile: exact TP × PP whole GPUs

Fitting published layout

Missing evidence / no fit

HTTP 200 may mix result statuses

3 Return a distinct API outcome

200 recommended

Profile ID, precision, backend, minimum
Explicit TP/PP + allocation; estimated: false
Evidence + runtime caveats in warning

200 no_recommendation

Missing capacity / node count / fitting profile
Explain why this entry cannot be sized
No suggested allocation for this entry

Request failed

400: invalid input / absent explicit type
401 / 403: access boundary rejected it
503: inventory or NGC retrieval failed

Current boundary

Sizing stops at the response. It does not deploy a model, reserve resources, check quota or automatically prefill a UI.
Installed capacity is not free capacity. Users or a separate integration may apply the suggestion through an existing deploy flow.

Blue: processing Green: successful result Amber: supported alternative Red: failure Gray: context

API decisions: GPU type, profiles and responses

POST /api/v1/inference/right-sizing/recommendation | Implemented request and response branches

Start with an authorized, valid request

Required: clusterId, image and exact tag. Optional: gpuType. Inventory is read completely before profile selection.
Invalid input -> 400. Missing credentials / denied permission -> 401 / 403 at the access boundary.

1 Select cluster hardware

gpuType omitted or null

Evaluate every distinct reported GPU type/capacity pair.
Empty inventory -> HTTP 200 with results: [].
There is no single global winner across GPU types.

gpuType supplied

Evaluate only the exact inventory GPU type, at each reported capacity.
Requested type absent -> HTTP 400.
Blank or whitespace-only type -> HTTP 400.

2 Fetch the model/tag metadata

Valid, nonempty selection

Read metadata cache / NGC

Exact image/tag cache: default 15 min.
Miss / cache failure -> fetch NGC.
Cache profiles only; inventory stays fresh.

Failure

503 dependency failure

Inventory or NGC retrieval failed.
Includes timeout or malformed data.
No partial recommendation response.

404

NGC 404 / no profiles

Continue with an empty profile set.
This is missing sizing information,
not a dependency failure.

3 Prepare each hardware entry

For each selected GPU type/capacity

Missing capacity or single-node count -> no_recommendation for this entry.
Resolve exact PCI identity; unknown / ambiguous names may use GPU-agnostic profiles.
Require profile ID, positive TP/PP and finite minimum; fit memory + one-node count, respect max_gpu_count.

4 Classify each eligible profile

First matching rule wins; evaluate all candidates

Direct PCI match?

gpu_device equals resolved PCI
Empty tested list is OK

No

Tested PCI match?

Resolved PCI appears in
tested_gpu_devices

No

GPU-agnostic?

gpu and gpu_device absent
Empty / missing tested list is OK

No

Skip this profile

Target does not match, or
only a GPU name is supplied.
Other profiles may still qualify.

Yes (Direct PCI match?)

Yes (Tested PCI match?)

Yes (GPU-agnostic?)

After all profiles (Skip this profile)

5 Select a fitting profile and return this entry

Choose the best fitting published profile

Fitting TP1 first. Otherwise parallel profiles: direct > tested > GPU-agnostic, then fewest GPUs.
Within the surviving tier/count: lowest per-device minimum, then profile ID. No scalar estimate.

Published layout selected

200 recommended

Profile ID, precision, backend, minimum and TP/PP
Exact published allocation + warning; estimated: false

Cannot size this entry

200 no_recommendation

Missing capacity / node count, no eligible minimum or no fitting profile.
Return a clear reason; no recommendation object.

A nonempty tested list is evidence, not an exclusive allowlist. Untargeted profiles may still be GPU-agnostic.
HTTP 200 can contain both recommended and no_recommendation entries. Empty inventory returns results: [] without an NGC lookup.

Allocation policy: minimum to resource request

After published-profile selection | TP1 sharing cutoff defaults to 90% | Decimal GB examples

1 Start with the selected profile and reported capacity

Published profile

Positive TP/PP, memory and profile ID.
Count = TP × PP, without rounding.
No invented extra headroom.

Reported hardware

Per-device capacity + one-node count.
Each profile must fit both.
Installed devices, not free capacity.

One sharing-policy value

WHOLE_GPU_CUTOFF_PERCENT
Default 90; valid range 1 through 100.
Invalid configuration is rejected.

2 Convert units and apply the cutoff

Use the published per-device minimum and exact layout

m = ceil(minimum GB x 1,000,000,000); C = capacity bytes. Reject profiles with m > C or count > one node.
TP1: compare m with ceil(C x cutoffPercent / 100). Parallel: allocate whole GPUs, regardless of cutoff.

3 Return one of three allocations

These ranges show the default 90% policy

TP1 below 90% of capacity

allocationType: gpu_memory; count: 1
gpuMemoryPerDeviceBytes = m
estimated: false

Example: 71.9 / 80 GB

Below the 72 GB threshold.
Request 71.9 GB of GPU memory.

TP1 at 90% through 100%

allocationType: gpu_count; count: 1
gpuMemoryPerDeviceBytes = C
estimated: false

Example: 72 / 80 GB

Exactly at the 90% threshold.
Request one whole GPU (80 GB).

Fitting parallel profile

allocationType: gpu_count; count: TP × PP
gpuMemoryPerDeviceBytes = C
estimated: false

Example: FP8 TP2 / PP1

57 GB floor; two 80 GB devices fit.
Request two whole GPUs on one node.

Meaning and limits

No fitting published profile or missing hardware evidence -> no_recommendation. No rescaling / power-of-two fallback.
The 90% cutoff is a sharing rule, not OOM protection. Exact counts do not guarantee runtime or interconnect compatibility.
Minima are publisher-defined; KV cache and runtime overhead settings may vary and require exact-profile confirmation.

Blue: processing Green: successful result Amber: supported alternative Red: failure Gray: context